MITS 5400G - Assignment 3 (Part 2) – 50%

Winter 2026

Deadline: By the end of March 26, 2026 (Thursday)

Compile your responses to the following tasks—including any:

- Source code
- Screenshots of terminal output (*Screenshots must be legible. If output is long, show the most relevant portion and note that it has been truncated*)
- Command you ran (as a screenshot or formatted code block)

into a **single PDF file** and submit it to Canvas.

Instructions

In this assignment, you will approach a dataset containing **2,000 structured SSH log entries** from **a real Linux system** that experienced active **brute-force** and **intrusion attempts** through the lens of AI-powered log analysis, using a real open-source LLM-based tool (llm-log-analyzer) backed by local Large Language Models running via Ollama. You will move from being a manual analyst to being an AI analyst: crafting intelligent prompts, running multiple models, comparing their outputs, and critically evaluating where AI excels and where it falls short.

Sample rows in dataset:

```
None
Dec 10 06:55:46 LabSZ sshd[24200]: Invalid user webmaster from 173.234.31.186
Dec 10 06:55:48 LabSZ sshd[24200]: Failed password for invalid user webmaster
from
173.234.31.186 port 38926 ssh2
Dec 10 07:07:38 LabSZ sshd[24206]: Invalid user test9 from 52.80.34.196
```

The tool:

The tool you will use is stratosphereips/llm-log-analyzer, developed at the Stratosphere Laboratory, Czech Technical University in Prague. It works by:

1. Reading a plain text log file
2. Reading a YAML prompt configuration file
3. Combining both and sending them as a prompt to a local Ollama LLM
4. Returning a structured analysis from the model

Full Command Line reference:

```shell
python log-analyzer.py -f <log_file> -c <prompt_yaml> [options]
```

Flag	Default	Description
-f	*(required)*	Path to the log file to analyze
-c	(required)	Path to the .yaml prompt config
-m	llama3.2	Ollama model name to use
-n	10	Number of lines to read from the end of the file
-o	log-analyzer-output.txt	File to save the LLM response to

Example commands:

```shell
# Basic run - last 10 lines with llama3.2
python log-analyzer.py -f openssh_converted.log -c my_prompt.yaml

# Run 50 lines with mistral, save output to a named file
python log-analyzer.py -f openssh_converted.log -c my_prompt.yaml -m mistral -n
50 -o mistral_output.txt

# Run 100 lines with phi3
python log-analyzer.py -f openssh_converted.log -c my_prompt.yaml -m phi3 -n
100 -o phi3_output.txt
```

Setup

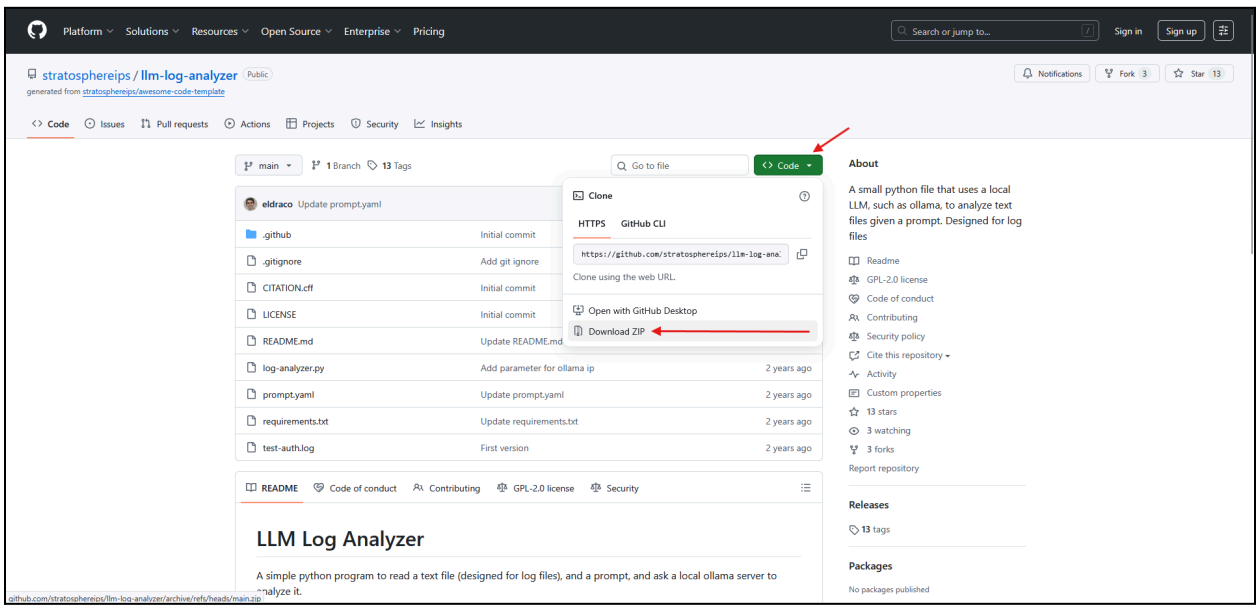
1. Install Python 3 (if not already installed)
2. Install Ollama (Go to https://ollama.com, download, and run the installer).
3. Open a terminal and run the following commands

```shell
# Start Ollama (it will run as a background service) - Leave this terminal
window open.
ollama serve
```

```
# Pull the required models
ollama pull llama3.2
ollama pull mistral
ollama pull phi3

# Verify the models are installed (You should see llama3.2, mistral, and phi3)
ollama list
```

4. Download or clone the tool's GitHub repository by visiting https://github.com/stratosphereips/llm-log-analyzer, clicking the green "Code" button, and selecting "Download ZIP."



5. Unzip the folder and open it in a terminal (using your preferred integrated development environment or command-line shell).
6. Create a Python virtual environment and install the required dependencies (*ensure you are inside the application directory in your terminal before running the commands*).

```shell
# On MAC OS run the following commands
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

```
# On Windows run the following commands
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

7. Run a sample command to verify that the tool is functioning correctly.

```Shell
python log-analyzer.py -f test-auth.log -c prompt.yaml
```

Questions:

Introductory

1. **Install Ollama and pull all three required models (`llama3.2`, `mistral`, `phi3`). In your report, include:**
   - A screenshot of `ollama list` showing all three models installed
   - The disk size of each model as shown in the list
2. **Run `log-analyzer.py` against the provided `test-auth.log` using the default `prompt.yaml` (both included in the codebase by default). *Ensure that all lines in the file are processed by setting the -n flag to 16*. In your report, include:**
   - The exact command you ran
   - A screenshot of the full terminal output (the LLM response)
   - A 3–5 sentence interpretation of what the model found, in your own words (not simply quoting the LLM)

Log Preparation

Objective: Convert the structured CSV dataset (Included in the Canvas dropbox) into a plain log file suitable for `log-analyzer.py`. This bridges your existing Python/Pandas skills into the AI analysis workflow.

You must reconstruct the original syslog format:

```
None
Dec 10 06:55:46 LabSZ sshd[24200]: Invalid user webmaster from 173.234.31.186
```

This is assembled from:

```
None
{Date} {Day} {Time} {Component} sshd[{Pid}]: {Content}
```

1. **Write a Python script that:**
    a. Loads OpenSSH_2k.log_structured.csv (available in Canvas) using Pandas
    b. Reconstructs each row into the syslog format above
    c. Writes all 2,000 lines to openssh_converted.log
    d. Prints a summary to the terminal: total lines converted, unique event types, most common event ID
    e. In your report include:
        i. Your complete Python script
        ii. A screenshot of the terminal output when the script runs
        iii. The first 5 lines of openssh_converted.log (paste as text, not screenshot)
2. **Before moving to AI analysis, perform a brief manual inspection of your converted log. Using Pandas (in the same script or a separate one), answer the following:**
    a. How many log entries contain the word "Invalid user" ?
    b. How many unique source IP addresses appear in those entries?
    c. What is the most frequently targeted invalid username?

Include your code and output in this report. This will serve as your baseline for comparison later on in Assignment 4.

Prompt Engineering

<u>Objective</u>: Learn that how you frame a question is just as important as the model you use. Write three targeted security prompts, execute them against the SSH log, and critically evaluate how differences in prompt design influence the LLM's output, accuracy, and depth of analysis.

Background - What is a prompt.yaml? The tool accepts a YAML configuration file containing an instruction context, followed by the log lines.

```
None
prompt: | You are a security analyst with deep knowledge of SSH authentication
systems. Analyze the following log entries carefully...
```

The full message sent to the LLM is:

```
None
<your prompt text>

<N lines of the log file>
```

Prompt quality directly determines the quality, specificity, and usefulness of the analysis you get back.

You are given three starter prompts (on Canvas) for the tasks below. The prompts are:

- `starter_anomaly.yaml` (for Anomaly & Timing Detection)
- `starter_brute_force.yaml` (for Brute Force Detection)
- `starter_threat_actor.yaml` (for Threat Actor Profiling)

1. **<u>Brute Force Detection Prompting</u>: Edit `starter_brute_force.yaml` to instruct the LLM (choose any of the three you have installed) to iteratively analyze the log file in chunks (e.g., a few hundred lines at a time) until the entire file has been processed. Initially, attempt to process up to the max 2000 lines in a single pass and evaluate whether this approach performs effectively before defaulting to smaller chunks, if needed. This prompt should instruct the LLM to:**

    a. Identify IP addresses attempting repeated failed logins
    b. Distinguish between failed password attempts and invalid username attempts
    c. List specific IPs it identifies as likely brute-force attackers
    d. Rate the severity of each identified actor (Low / Medium / High / Critical)
    e. Explicitly not offer remediation advice (analysis only)

    In your report include:

    - The command(s) you ran to achieve this
    - Your complete starter_brute_force_prompt.yaml content
    - Screenshot of the LLM output
    - 4–6 sentences analyzing: Was the output useful? Was it specific? What did it get right or wrong?

2. **<u>Anomaly & Timing Detection Prompting</u>: Edit `starter_anomaly.yaml` to instruct the LLM (choose any of the three you have installed) to iteratively analyze the log file in chunks (e.g., a few hundred lines at a time) until the entire file has been processed. Initially, attempt to process up to the max 2000 lines in a single pass and evaluate whether this approach performs effectively before defaulting to smaller chunks, if needed. This prompt should instruct the LLM to:**
    a. Look for rapid repeated attempts within short time windows
    b. Identify any unusual sequences of events (e.g., immediate disconnect after auth failure)
    c. Flag any events that stand out as anomalous compared to the rest of the log
    d. Report specific timestamps where anomalous clusters occur

    In your report include:

    - The command(s) you ran to achieve this
    - Your complete starter_anomaly.yaml content
    - Screenshot of the LLM output
    - 4–6 sentences analyzing: Did the model identify timing patterns? Were the anomalies it flagged genuine?

3. **<u>Threat Actor Profiling Prompting</u>: Edit `starter_threat_actor.yaml` to instruct the LLM (choose any of the three you have installed) to iteratively analyze the log file in chunks (e.g., a few hundred lines at a time) until the entire file has been processed. Initially, attempt to process up to the max 2000 lines in a single pass and evaluate whether this approach performs effectively before defaulting to smaller chunks, if needed. This prompt should instruct the LLM to:**
    - Group activity by source IP address
    - For each unique attacker IP: describe what they attempted, how many times, and when
    - Characterize the attack style (opportunistic scan vs targeted attack vs credential stuffing)
    - Produce a structured output format, e.g., one section per IP

    In your report include:

    - The command(s) you ran to achieve this
    - Your complete starter_threat_actor.yaml content
    - Screenshot of the LLM output
    - 4–6 sentences analyzing: How well did the model group and characterize the attacker's behavior?

4. **Reflection - Prompt Engineering (Maximum 400 words)**:
   Looking across your three prompts and their outputs: what patterns did you notice about what makes a security prompt effective? What prompt design choices produced the most actionable output, and which produced vague or generic responses?
5. **References**
   All sources must be properly cited and referenced in American Psychological Association (**APA) format**. https://apastyle.apa.org/style-grammar-guidelines/paper-format
6. **Use of Generative AI Tools**
   You may use AI tools for assistance; however, submissions must reflect **your own analysis, writing style, and critical thinking**. Do not copy AI-generated content verbatim—ensure the work is original, appropriately adapted, and fact-checked. 😊
7. **Disclose use of Generative AI tools**
   "Use of Generative AI Tools: Portions of this work involved the use of generative AI tools (e.g., name them) for code debugging assistance and grammar refinement, [something else]. All conceptual contributions, system design, algorithm development, implementation, and final editorial decisions [verify] were performed by the student. The use of AI tools did not generate original content, data, or analysis [verify]."

******* END *******